2012 IEEE Fifth International Conference on Cloud Computing

Efficient Map/Reduce-based DBSCAN Algorithm with Optimized Data Partition

Bi-Ru Dai
Department of Computer Science and Information Engineering,
National Taiwan University of Science and Technology, Taipei, Taiwan, ROC.
Email: brdai@csie.ntust.edu.tw

I-Chang Lin
Department of Computer Science and Information Engineering,
National Taiwan University of Science and Technology, Taipei, Taiwan, ROC.
Email: M9915018@mail.ntust.edu.tw

Abstract—DBSCAN is a well-known algorithm for density-based clustering because it can identify the groups of arbitrary shapes and deal with noisy datasets. However, with the increasing amount of data, DBSCAN algorithm running on a single machine has to face the scalability problem. In this paper, we propose a Map/Reduce-based DBSCAN algorithm called DBSCAN-MR to solve the scalability problem. In DBSCAN-MR, the input dataset is partitioned into smaller parts and then parallel processed on the Hadoop platform. However, choosing different partition mechanisms will affect the execution efficiency and load balance of each node. Therefore, we propose a method, partition with reduce boundary points (PRBP), to select partition boundaries based on the distribution of data points. Our experimental results show that DBSCAN-MR with the design of PRBP has higher efficiency and scalability than competitors.

Keywords-data mining; data clustering; DBSCAN; Map/Reduce; data partition; Hadoop; cloud computing.

I. INTRODUCTION

In recent years, various types of spatial data such as environmental assessments, traffic services, meteorological conditions, GPS, geo-tagged images emerge. People use location-acquisition technology to locate their positions and use the Internet to share large amount of these spatial data. Therefore, how to find out valuable information from these datasets becomes an important issue.

Data mining, which is a major technology for discovering hidden knowledge from large databases [1] [2] [3], attracts lots of research attention. Discovering relationships and group behaviors of the data is important task to provide useful information for decision making, such as climate distribution, metropolis plan, census, etc. Clustering [4] is a very useful unsupervised learning technique of data mining. Clustering techniques partition data points into a number of groups such that the data points in the same group are similar. There techniques are extensively used in many areas such as bioinformatics, marketing, astronomy, pattern recognition, image processing, etc. However, with the increasing amount of data, working with single processor is inefficient. Traditional algorithms running on a single machine face the scalability problems, thus many researchers start to find solutions by cloud computing techniques [5][6][7].

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) [8] is one of major techniques in clustering algorithms. It is popular because of the ability of discovering clusters with arbitrary shapes for providing much interesting information. However, when it is applied on large databases, the problems of scalability and execution complexity are still big challenges. Thus, many existing studies try to improve the efficiency of DBSCAN algorithm. For example, TI-DBSCAN [9] uses the triangle inequality property to quickly reduce the neighborhood search space without using spatial indices. The work in [10] enhances DBSCAN by first using CLARANS [11] to partitioning the dataset for reducing the search space of each partition instead of scanning the whole dataset. GRIDBSCAN [12] constructs a grid that allocates the data points into similar partitions and then DBSCAN processes each partition separately. These algorithms improve the efficiency of DBSCAN, but are still not efficient enough to process massive data. Therefore, we propose a distributed mining algorithm for DBSCAN to address the scalability problem. The proposed algorithm DBSCAN-MR, which stands for distributed DBSCAN with Map/Reduce, is designed on the Hadoop platform [13], which uses Google's Map/Reduce-style [14]. Nevertheless, there are some challenges when DBSCAN is designed with the Map/Reduce structure. First, previous works on distributed system query a global spatial index to obtain correct global results, but this approach is not suitable for the Map/Reduce-style system because it incurs lots of inter-node communication. Second, the load of each node needs to be balanced, or the efficiency of the entire system will be reduced.

In this paper, we address the above challenges to design a Map/Reduce-style algorithm which uses a distributed index and optimizes load balance and execution efficiency. The contributions of this paper are as follows. First, we propose the DBSCAN-MR algorithm, which is a Map/Reduce-style algorithm for DBSCAN. It is a parallel processing approach which can be executed on the cloud and does not require a global index at all. In addition, we propose the partition with reduced boundary points (PRBP) algorithm to optimize data

978-0-7695-4755-8/12 $26.00 © 2012 IEEE
DOI 10.1109/CLOUD.2012.42

IEEE computer society

Authorized licensed use limited to: UEL - Universidade Estadual de Londrina. Downloaded on October 21,2026 at 20:04:58 UTC from IEEE Xplore. Restrictions apply.

partition by considering the distribution of clusters. It can be used to reduce the load of each node and enhance the efficiency of the entire framework. With the optimized partition method and the concept of cloud computing techniques, DBSCAN-MR can deal with huge amounts of data. The experimental results verify the high efficiency and scalability of DBSCAN-MR and PRBP.

The remainder of this paper is organized as follows. We discuss several clustering algorithms related to our work in Section 2. In Section 3, we introduce the proposed algorithms DBSCAN-MR and PRBP. Experimental evaluations are shown in Section 4. The conclusions are given in Section 5.

II. Related Work

The density-based idea is one of the major approaches for clustering algorithms. It is based on the idea that the data points which form a dense region should be grouped together into one cluster. They use a fixed threshold value to determine dense regions. They search for regions of high density in a feature space that are separated by regions of lower density. DBSCAN [8], OPTICS [15], DENCLUE [16], CURD [17] are popular density-based clustering algorithms. In our study, we choose DBSCAN algorithm because it does not only availably avoids noises but also effectually processes various datasets. However, DBSCAN is still not efficient enough for the massive dataset. In order to lower the time complexity, the academia has presented a grid-based clustering technique [8], which divides the data space into disjunctive grid. The data points in the same grid can be treated as a unitary object, such that all the operations of clustering are applied to the grids instead of the points.

Nevertheless, with the increasing amount of data, DBSCAN running on single machine still meets the bottleneck and the effective is fallen. Therefore, many researchers work on distributed and parallel data mining algorithms [18][19][20]. The cloud computing technology is used to address with huge amount of data. Hadoop [13] is an open source project aiming at building a cloud infrastructure which is designed with Google's Map/Reduce-style [14].

In this paper, we design an efficient distributed algorithm based on the cloud computing technology to improve the scalability of DBSCAN. Before presenting our methods, the concepts of DBSCAN and Map/Reduce-style design are briefly introduced in Section 2.1 and Section 2.2 respectively.

2.1. DBSCAN

The major idea of density-based clustering is that: given radius ε (*Eps*), each object of a cluster has to contain at least a minimum number (*MinPts*) of neighborhoods, i.e. the cardinality of the neighborhood has to exceed a threshold. The formal definitions for this notion of a clustering is defined in [8]. DBSCAN checks the Eps-neighborhood of each point in the database. If the Eps-neighborhood of p NEps(p) has more points than *MinPts*, a new cluster C containing the points in NEps(p) is created. Next, each point q in C which has not yet been processed is checked. If NEps(q) contains points more than *MinPts*, each neighborhood of q which is not contained in C is added to the C. This procedure is repeated until no new point can be added to current cluster C. DBSCAN repeats above steps until all points are processed.

2.2 Map/Reduce

The Map/Reduce framework [13] is based on two primitives, Map and Reduce, from functional programming. They are defined as follows:

$$\text{Map: } (k1, v1) \rightarrow (k2, v2)$$
$$\text{Reduce: } (k2, v2) \rightarrow (k2, v3)$$

The Map function consists of a list of key/value pairs and outputs a list of intermediate key/value pairs (k2, v2). The Reduce function takes all values associated with the same key and produces a list of key/value pairs. The sorted output of the reducers is the final result of the Map/Reduce process. Programmers implement the application logic using these two primitives. The parallel execution of each primitive is managed by the system runtime. As such, developers only need to design a series of Map/Reduce jobs to split data and merge results.

III. Distributed Density-Based Clustering with MapReduce

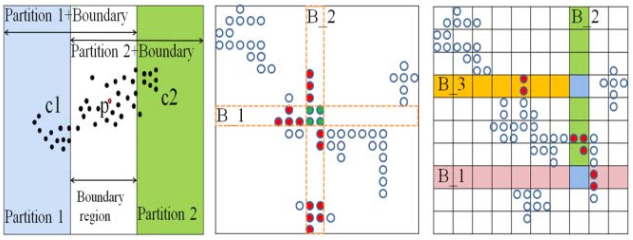
In this section, we introduce our idea which is based on the Hadoop platform. We design a distributed algorithm for a popular density-based clustering algorithm DBSCAN [8]. The proposed algorithm is called distributed DBSCAN with Map/Reduce and abbreviated as DBSCAN-MR. It improves the scalability of DBSCAN by dividing the input data into smaller parts and sending them to the nodes on the cloud for parallel processing. Load balance of each node and minimization of the total execution time are the major issues of this framework. To achieve these goals, we devise mechanisms to conquer several important challenges related to data partition and the design of Map/Reduce-style DBSCAN algorithm.

First, the data set should be partitioned and distributed to different nodes for processing on the cloud environment. Each node clusters a subset of the original data separately. Nevertheless, the traditional DBSCAN algorithm uses a global index structure which requires extensive inter-node communication and is therefore unsuitable for the Hadoop platforms. In this paper, we use a distributed index to handle this problem. We design parallel processing approach which does not require a global index at all.

Second, data points of the same cluster are probably scattered among different nodes. When the dataset is partitioned, data points around the boundary should be duplicated in order to merge the clusters scattered in

Authorized licensed use limited to: UEL - Universidade Estadual de Londrina. Downloaded on October 21,2026 at 20:04:58 UTC from IEEE Xplore. Restrictions apply.

different nodes. Note that the number of extra boundary points will affect the efficiency of the clustering step in each node and the step of merging clustering results from different nodes. For example, as shown in Figure 1 (a), the dataset is divided into two parts, partition 1 (blue) and partition 2 (green), and each part is extended to include the boundary region (white). When these two clusters from different partitions, such as c1 and c2, are extended to the same boundary region, data points in the boundary region can help us determine whether these two clusters should be merged or not. However, boundary points should be copied and put into all adjacent partitions and this increases the load of nodes. Another example as shown in Figure 1 (b) and Figure 1 (c), where different partition positions are selected, we can observe that the number of boundary points in Figure 1 (b) is more than that in Figure 1 (c) when the input data set is divided into four partitions. In Figure 1 (b), the data set is first divided by spilt region *B*_1, and the number of boundary points generated is twice of the number of points in the region (8 * 2 = 16). Then the data set is divided by spilt region *B*_2 and points in *B*_2 region are doubled to form the boundary points (14 * 2 = 28). So there are 44 (16 + 28) boundary points in Figure 2 (b). In contrast, the number of boundary points in Figure 1 (c) is 14, which can be calculated similarly (2 * 2 (from *B*_1) + 3 * 2 (from *B*_2) + 2 * 2 (from *B*_3) = 4 + 6 + 4 = 14). This example illustrates that different partition approaches generate different amount of boundary points. Note that massive boundary points affect the execution efficiency because these boundary points not only increase the load of each node but also increase the time for merging results from different nodes. In addition, the load balance of each node is an important concern for designing the partition method. Load imbalance negates the benefits of parallelism. Worse, the whole Map/Reduce job fails when a node runs out of memory. To achieve load balance and to improve the entire performance of the framework, we have to ensure that each node will not run out of memory and generates the minimum number of boundary points. To achieve these goals, we design an approach called *partition with reduced boundary points* (PRBP).



(a) (b) (c)

Figure 1.The examples of boundary regions. (a) The dataset is divided into two main parts, partition 1 (blue) and partition 2 (green), where both partitions are extended by the boundary region (white). Cluster c1 in partition 1 and cluster c2 in partition 2 can be identified as a single cluster by the data points in the boundary region. The examples in (b), (c) illustrate the results of difference partition methods when the input dataset is divided into four partitions. The total number of boundary points generated by (b) and (c) are different. Thus, careful design of partition method can reduce the amount of boundary points and improve the efficiency of the following steps.

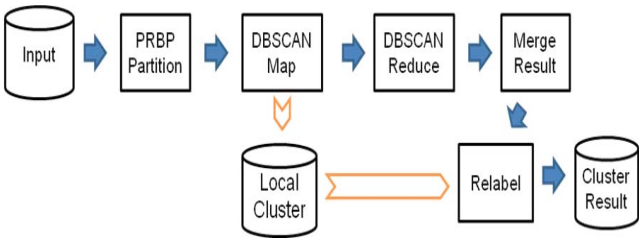


Figure 2.The framework of DBSCAN-MR. DBSCAN-MR runs in five phases. Each phase exchanges data in the form of standard relation or key-value pairs. Thus, DBSCAN-MR can be completely designed in Map/Reduce.

As shown in the Figure 2, our DBSCAN-MR algorithm includes five phases: In the first phase (*partition with reduced boundary points*), the dataset is partitioned with the consideration of not only maintaining the load balance between nodes but also minimizing the number of boundary points in order to increase the efficiency of clustering and merging. In the second phase (DBSCAN-Map), DBSCAN is executed on each node using the kd-tree [21] space index (kdDBSCAN) locally on its assigned data partition. In the third phase (DBSCAN-Reduce), we find out the same point indexes between partitions and get the clustering ID (CID) of these points. The fourth phase (Merge result) merges the results from DBSCAN-Reduce to discover the global structure of clusters based on the boundary points. Finally, in the Relabel phase, local clustering results from each partition are relabeled to identify the global clusters. Since phases exchange information in the form of standard relations or key-value pairs, the algorithm can be naturally expressed in various Map/Reduce-style frameworks. The details of each phase are introduced in the following subsections respectively. In Section 3.1, the pre-processing of DBSCAN-MR for optimized data partition is introduced. The partitioned data are locally clustered with DBSCAN-Map, and boundary points are assembled with DBSCAN-Reduce, which are presented in Section 3.2 and Section 3.3 respectively. In Section 3.4, the output clusters scattered in different partitions of DBSCAN-Reduce are combined. The approach how all data points are relabeled with their global clustering ID is described in Section 3.5.

3.1 Partition with reduced boundary points

In order to achieve load balance of each node and improve the execution efficiency of the whole algorithm, we devise a splitting process to select the best split regions. Our proposed partition algorithm is called *partition with reduced boundary points*, abbreviated as PRBP. The split region, also called partition boundary, is the region between adjacent partitions. The data points in the split region are called boundary points, and they are added into both partitions for discovering connected clusters in different partitions. The goal of this step is to minimize the total number of points in partition boundaries. First, each

Authorized licensed use limited to: UEL - Universidade Estadual de Londrina. Downloaded on October 21,2026 at 20:04:58 UTC from IEEE Xplore. Restrictions apply.

dimension is divided into slice with equal width and the data distribution is calculated. The setting of slice width will be discussed later. In addition, two parameters, θ and β, are provided to prevent unbalanced partitions and to avoid running out of memory in any node respectively. Then, the split regions are selected according to the data distribution iteratively. Detailed steps of this algorithm are described below.

The PRBP Algorithm, as shown in Algorithm 1, contains three steps: (1) initializing slices for each dimension, (2) calculating accumulative points for each successive slice, and (3) selecting the best slice to partition.

Step 1- initializing slices for each dimension: In this phase, all data points are sorted in ascending order according to each attribute value, respectively, where we get a sorted list for each dimension. Then, it builds a set of successive slice by the **buildSliceUse2Eps** method, which is explained in detail as follow.

BuildSliceUse2Eps method: First, it constructs a gird which its width of slice is 2ε for each dimension. Figure 3 (a) shows an example. Choosing 2ε to be the width of slice minimize the number of boundary points because DBSCAN extends the cluster within the ε radius, where we need at least 2ε wide boundary region to contain enough information for merger.

Step 2- calculating accumulative points for each successive slice: In this phase, each slice accumulative calculates the total number of points in it. Then it sets the search range R to be (total number of points) * $\theta < R <$ (total number of points)* (1 - θ), where θ must be a value in the range $0 < \theta < 0.5$. This range R is for the next phase to select the best slice which can achieve the load balance.

Step 3- selecting the best slice to partition: In this phase, it checks each partition p of partition set P. For each partition p, **partitionUseBestSlice** method which is explained below is executed until all partition p of partition set P are processed. When a partition p has to be split to two parts $p1$ and $p2$, it removes the original partition p from P and puts $p1$ and $p2$ into P.

Algorithm Partition with reduced boundary points (D, Eps, β, θ)

Input: D: dataset;
{**Step I:** initializing slices for each dimension}
1. p=**buildSliceUse2Eps**(D,Eps, β, θ);
2. P.add(p); // P is a set of partitions
{**Step II:** calculating accumulative points for each successive slice}
3. **For** each dimension d in p **do**
4. **For** each slice s in d **do**
5. calculates the number of points *s.count* and the accumulative number of points *s.total*
6. **End for**
{**Step III:** selecting the best slice to partition data}
7. **For** each partition p in P **do**
8. **If** p is not processed then
9. **If partitionUseBestSlice**(p, β, θ) is true // if return value is true, p is split to two part in **partitionUseBestSlice**
10. delete p from P;
11. **End if**
12. **End if**
13. **End for**
14. **Return** P

Algorithm 1 Partition with reduced boundary points

Algorithm partitionUseBestSlice(p, β, θ)

Input: p: a partition
tmp_min = ∞; // This variable stores the minimum number of point in a slice
tmp_d_index = null; *tmp_slice_index* = null; // partition indexes of the slice
1. **If** *p*.size < β **do** // if p contains less than β points,
2. **Return** false; // it does not need to be partitioned
3. **End if**
4. **Else** // find out the best slice
5. **For** each dimension d in p **do**
6. **If** *d.sliceCount* < 3 **do** // the number of slices in a dimension
7. **Return** false; // must be larger than 3
8. **Else**
9. **For** s = 1 to n-2 slice s in d **do** // to avoid selecting the boundary slices
10. **If** *p.size* * θ < *s.total* < *p.size* * (1 - θ) **do**
11. **If** *s.total* < min **do**
12. *tmp_d_index* = *d.index*; *tmp_slice_index* = *s.index*; *tmp_min* = *s.count*;
13. **End if**
14. **End if**
15. **End for**
16. **End for**
17. **End else**
18. P.add(**partitionDataUsebestSlice**(*tmp_d_index, tmp_slice_index*));
// using the best slice to partition the data into two new partitions
19. **Return** true;

Algorithm 2 partitionUseBestSlice

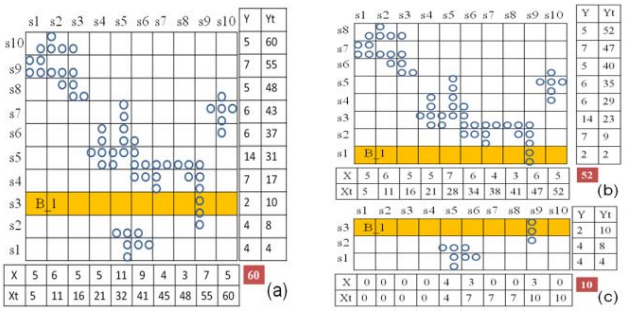


Figure 3.Illustration of data partition. (a) For each dimension, data points are divided into slices with equal width. The number of data points in each slice and the accumulated number of data points from the first slice to the current slice are recorded for each slice. The slice (Y_s_3=2) with the least number of data points among all dimensions will be selected as the split region (B_1). (b) and (c) show the partition results of this split step. The information of each slice is updated for further splitting.

PartitionUseBestSlice method: The objective of the **PartitionUseBestSlice** job is to find out the best partition slice from all possible partition slices obtained by Step 1. We can recursively split the space until the estimated data size of each partition fits in the memory space of nodes, thus we can avoid running out of memory problem. First, we check the number of the data points if it less than the threshold β. A partition which its size is smaller than β does not need to be partitioned anymore. Next, it checks the number of possible slices of each dimension in this partition p. The number of slices must be larger than 3 because a dimension which only has 3 or less slices does not wide enough to partition. Then, it searches each available slice of each dimension, where the points of the slice have to be in the range R for achieving better load balance. It chooses the slice which has the least points, stores its slice index, dimension index and number of points of slice. Finally, we use **partitionDataUseBestSlice** method according to the stored indexes to finish the partition. An example is shown in Figure 3. We find out the best slice Y_s3 which is available and has the least points in it, and

Authorized licensed use limited to: UEL - Universidade Estadual de Londrina. Downloaded on October 21,2026 at 20:04:58 UTC from IEEE Xplore. Restrictions apply.

then partition data by this slice into two partitions as shown in Figure 3(b) and 3(c).

3.2 DBSCAN-Map

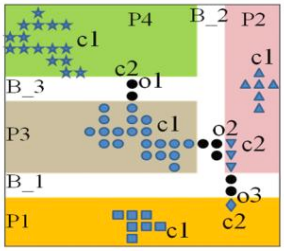
The partition result of algorithm PRBP will be sent to the Hadoop distributed file system where each partition is regarded as an input of a node. For each node, DBSCAN algorithm is executed on the assigned partition. Note that each partition is extended to include the boundary points around the partition. In addition, the efficiency of the original DBSCAN algorithm is improved from $O(n^2)$ to O(nlogn) by using the KD-Tree [21] spatial index.

Algorithm DBSCAN-Map(partition p)

Input : *p*: a partition
Var *p*= read input data;
1. **KD**=build_spatial_index(*p*); // building the KDTree spatial index
2. **KD.DBSCANClustering**(*p*); // running DBSCAN on *p* with the KDTree index
3. **For** each point *Pts* in *p* **do**
4. **If** *Pts.isboundary* **do** // storing the result of boundary points to HDFS
5. Output(*Pts*.index, *partition*.index + *Pts.cid* + *Pts*.iscore);
6. **End if**
7. **Else** // storing result of other points to local disk
8. writeLocal(*Pts*.index, *partition*.index + *Pts.cid*);
9. **End else**
10. **End for**

Algorithm 3 DBSCAN-Map

As shown in Algorithm 3, it first builds the KD-Tree spatial index from the input partition *p* for DBSCAN. After executing DBSCAN algorithm on partition *p*, the results can be divided into two parts, local region and boundary region. As shown in Figure 4, the boundary results are stored as (*point_index*, *partition_index*+CID+*isCore*(true or false)) in the Hadoop distributed file system (HDFS) [13] and used as the input of DBSCAN-Reduce. The local results are stored as (*point_index*, *partition_index*+CID) in local disk. Note that *isCore* is a flag to identify whether a data point is a core point or not in the DBSCAN clustering process, and CID is clustering identification number for DBSCAN clustering results.



(a)

Local(P1,P2,P3,P4)	
key	value
P1 point index	P1c1 or P1c2
P2 point index	P2c1 or P2c2
P3 point index	P3c1
P4 point index	P4c1

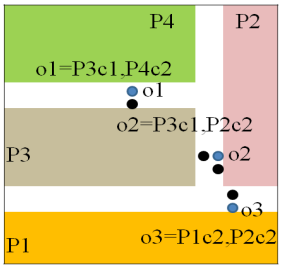
Boundary(B_1,B_2,B_3)		
area	key	value
P3	o1	P3c1,isCore
P4	o1	P4c2,isCore
P2	o2	P2c2,isCore
P3	o2	P3c1,isCore
P1	o3	P1c2,isCore
P2	o3	P2c2,isCore
:	:	:

(b)

Figure 4.The clustering result of DBSCAN-Map. In (a), the dataset is divided into four parts P1, P2, P3, and P4, and then they are clustered by DBSCAN-Map separately. Local clustering results P1c1, P2c1, P3c1 and P4c1 are generated and stored into local region and boundary region as shown in (b).The local part is stored in local disk while the boundary region is sent to HDFS system as the input data of DBSCAN-Reduce.

3.3 DBSCAN-Reduce

In this phase, DBSCAN-Reduce collects boundary results from DBSCAN-Map. As shown in Figure 5, data points with the same point index are gathered together from different partitions. If the point is a core point in any partition, the flag of *isCore* is set as true, which means that this point belongs to a cluster and is a core point in at least one partition. Such points can help us to identify if a cluster is scattered in different partitions and should be merged together. Note that the inputs which have the same key are executed at the same reducer task.



(a)

Point index	Cluster ID list
o1	P3c1,P4c2
o2	P2c2,P3c1
:	:
o3	P1c2,P2c2

Reduce output

key	Value
o1	{P3c1,P4c2}+isCore
o2	{P2c2,P3c1}+isCore
:	:
o3	{P1c2,P2c2}+isCore

(b)

Figure 5.The input data and output results of DBSCAN-Reduce. (a) Only boundary points from DBSCAN-Map results should be sent to DBSCAN-Reduce, as shown in the white region. DBSCAN-Reduce will collect and combine the records with the same point index into a new list as (b). For example, point o1 has two records originally generated by different partitions, P4c2 and P3c1. They are combined together as a single entry {P3c1, P4c2} in the new list.

Algorithm DBSCAN-Reduce(key, values)

Input: *key*, *value*; map output pair {*point_index*, *partition.index*+*Pts.cid*+*Pts.isCore*}
Var *cid_list* = null; // set to store cluster IDs for the same index point
Var *merge* = false;
1. **For** each point *Pts* in values **do** // values is a set of CID for the same // indexes of points
2. **If** *Pts*.iscore **do** // if point *Pts* is a core point
3. *merge* = true;
4. **End if**
5. **If** !*cid_list.contains*(*Pts.cid*) and *Pts.cid* != noise **do**
6. *Pts*.cid and *cid_list* are combined into *cid_list*
7. **End if**
8. **End for**
9. **If** *merge* == true **do** // storing the result to HDFS
10. Output(*key*, *cid_list* + true) // *key* is point index
11. **End if**
12. **Else**
13. Output(*key*, *cid_list* + false)
14. **End else**

Algorithm 4 DBSCAN-Reduce

The pseudocode is shown in Algorithm 4. It checks each input point whether it is a core point in any partition. If yes, it is tagged *isCore*=true and is added to group list. When this checking process is done, the group list is outputted as Figure 5(b).

3.4 Merge result

To identify a cluster that spans multiple partitions, points in the partition boundaries should be examined. We use the output of DBSCAN-Reduce as a combination list of clusters. Each combination indicates the clusters which must be merged.

Algorithm 5, which we call merge boundary result, shows the detailed pseudocode of the merge procedure. First, it generates a list which stores the cluster IDs which need to be merged according to the output of DBSCAN-Reduce. The output of DBSCAN-Reduce is a set of points, where each point can be labeled with one or more cluster IDs. For each point labeled with more than one cluster IDs and is

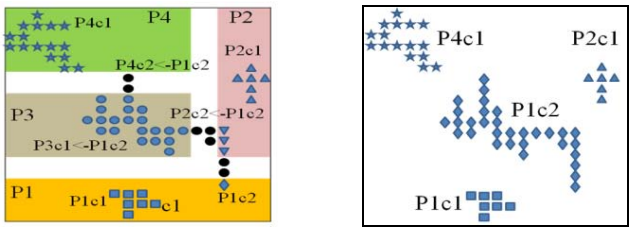
Authorized licensed use limited to: UEL - Universidade Estadual de Londrina. Downloaded on October 21,2026 at 20:04:58 UTC from IEEE Xplore. Restrictions apply.

tagged *isCore*=true, the clusters which these cluster IDs represent need to be merged, thus these clusters will be relabeled in the next phase. The output of this phase is a set of merge list *M* which is a mapping between the pre-merge cluster IDs and the post-merge cluster IDs. For example in Figure 6, there are three elements {{P1c2, P2c2}, {P2c2, P3c1}, {P3c1, P4c2}} in the merge list *M* of the dataset. This set *M*, however, is not yet quite complete, as there are potentially clusters that should be further combined. In fact, P1c2, P2c2, P3c1 and P4c2 need to be merged together, but if we relabel P2c2 to P1c2, P3c1 to P2c2 and P4c2 to P3c1, we will get three clusters P1c2, P2c2 and P3c1. We can infer such missing links by examining the pair wise inter-sections between sets of merged cluster IDs. Since {P1c2, P2c2} and {P2c2, P3c1} both contain P2c2, {P2c2, P3c1} and {P3c1, P4c2} both contain P3c1, the merge list can be combined to {P1c2, P2c2, P3c1, P4c2}. In the last step, the algorithm simply sorts the cluster ID in rise order and relabels all clusters in the list as the first one. In the example in Figure 6, these four clusters {P1c2, P2c2, P3c1, P4c2} are relabeled to P1c2 and become a single cluster.

Algorithm Merge boundary result (key, value)

Input: DBSCAN-Reduce
output: *M* // *M* is merge list
Var *M* = null; // set to store cluster IDs of merged clusters
1. data=Hdfs.open(DBSCAN-Reduce output);
2. **For** each point *Pts* in data **do**
3. **If** *Pts.iscore* **do**
4. Finding out all merge combination of *Pts* and put them into merge list set *M*
5. **End if**
6. Filtering the same combination from *M*
7. **End for**
8. **repeat**
9. **For** each *C1, C2* ∈ *M* **do** // C1 and C2 are merge combinations
10. if C1∩C2 != Ø then combining *C1* and *C2*
11. **End for**
12. **Until** all merge combinations in *M* do not change anymore
13. **For** each *C* ∈ *M* **do** // For example, {c1,c4,c3} is sorted as {c1,c3,c4}
14. Sorting *C* in the rise order by CID
15. **End for**
16. **Return** *M*

Algorithm 5 Merge boundary result



(a) (b)

Figure 6.(a) The merge result by combining the merge list produced from DBSCAN-Reduce. Finally, all data points are relabeled by the correct global CID according to the merge results, as shown in (b).

3.5 Relabel Data Points

In DBSCAN-MR, there are two phases of relabeling, boundary and global. In last phase, Boundary relabeling allocates each the same point ID and the complete merge list is built based on the output of Merge boundary. In the final phase global relabeling, all data points are relabeled according to the complete merge list. It updates each local result to the correct result of clustering as show in Figure 6 (a) and (b).

IV. PERFORMANCE EVALUATION

In this section, we conduct several experiments to evaluate the performance of DBSCAN-MR and the effects of input parameters. Two algorithms are selected to be compared with DBSCAN-MR. First, we implement algorithm GRIDBSCAN [12] in Map/Reduce for comparison. GRIDBSCAN enhances the efficiency of DBSCAN by constructing a grid surrounding data space, partitioning data into cells which the best width is =10ε, applying DBSCAN on each partition, and finally merging the resulting clusters to produce the true clustering. The second one is DBSCAN-MR-N, which is a variation of DBSCAN-MR without storing the slice information. In other words, the **buildSliceUse2eps** method is executed for selecting each split region. Each partition data may have different distribution, but DBSCAN-MR-N can still find out the best slice because it recalculate the precise information each time. DBSCAN-MR-N may obtain boundary points less than DBSCAN-MR, but reprocess needs to increase execute times for split processing. DBSCAN-MR is designed with Java and runs on top of Hadoop version 0.20.2. The used Hadoop cluster consists of 10 nodes and each node contains 4 intel Xeon(TM**)** CPU 3.00GHz and 4GB RAM running Cent OS 6.0 Linux operating system. Note that the clustering results of DBSCAN-MR are exactly the same as that of the original DBSCAN for the same parameters (*MinPts*, ε), which means that DBSCAN-MR successfully performs the clustering process of DBSCAN with the cloud computing technology.

The datasets used in our experiments are described in Section 4.1. Experimental results and discussions are presented in Section 4.2.

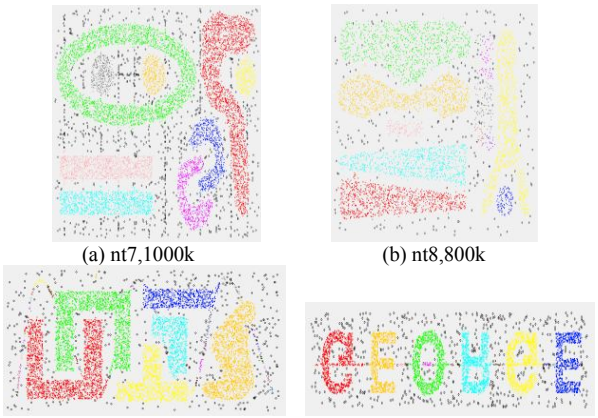
4.1 Experimental Designs

We use four synthetic datasets and one real dataset to illustrate the performance of our algorithm. The information of each dataset is briefly summarized below.

Synthetic datasets: t7,10k.dat, t4,8k.dat, t5,8k.dat and t8,8k.dat, are originally used in *Chameleon* [22]. However, the sizes of these datasets are too small (10k and 8k objects) to show the efficiency for cloud computing. Therefore, we generate larger synthetic datasets, called nt7,1000k, nt4,800k, nt5,800k and nt8,800k, based on the features of these four datasets. As shown in Figure 7, data set nt7,1000k contains 1000k objects, while nt5,800k, nt4,800k and nt5,800k contain 800k objects respectively.

Real dataset: The data is California space data which feather contains geographic information, population, ecology, and management of public lands from USGS geoname datasets [23]. In addition, we also produce new synthetic datasets-California780k by data feature for real data (53281) of California. It contains feature information of 785685 objects.

Authorized licensed use limited to: UEL - Universidade Estadual de Londrina. Downloaded on October 21,2026 at 20:04:58 UTC from IEEE Xplore. Restrictions apply.



(a) nt7,1000k (b) nt8,800k

(c) nt4,800k (d) nt5,800k

Figure 7.The clustering results of DBSCAN-MR for four synthetic datasets.



(a) (b) (c)

Figure 8.The partition results of dataset nt7,1000k generated by (a) GRIDBSCAN, (b) DBSCAN-MR-N and (c) DBSCAN-MR. Points with black color are boundary points between partitions.

	boundary points	partition time(s)	MapReduce time(s)	Merge time(s)	Total run time(s)
GRIDBSCAN	328900	5.991	130.56	5.655	142.116
DBSCAN-MR-N	114600	12	101.405	2.466	115.951
DBSCAN-MR	114700	7.953	99.221	2.497	109.671

Table 1.The number of boundary points and execution time of clustering the dataset nt7,1000k with four node by difference algorithms.

4.2 Experimental Results

In this section, we first illustrate the performance of different partition methods as shown in Figure 8. The width of grid cell is set as $10*\varepsilon$ for GRIDBSCAN, which is the best grid cell width proposed in [12]. Different partitions are marked in different colors, and boundary points are marked in black. The number of boundary points and the execution time of major phases of each algorithm are summarized in Table 1. We can observe that GRIDBSCAN generates much more boundary points (328900) than DBSCAN-MR-N and DBSCAN-MR (114600 and 114700). The massive boundary points increase the load of each node and incur unnecessary map jobs which decrease the efficiency. Note that each map job requires time for initialization and inter-node communication; hence GRIDBSCAN spends more time in Map/Reduce and Merge phases. Although the partition method of GRIDBSCAN is simple and fast, it generates too many boundary points which increase the execution time of following phases. However, our partition algorithm PRBP takes the data distribution into consideration. Therefore, better partition boundaries can be selected with slight costs of execution time. Besides, we can discover DBSCAN-MR-N costs more than DBSCAN-MR in the partition job since DBSCAN-MR-N can reprocess **buildSliceUse2eps** in partition job. DBSCAN-MR-N produces less boundary points than DBSCAN-MR by the reprocessing, but it needs more execution time on partition by the same reason. Consequently, DBSCAN-MR is more efficient on the total running time at five synthetic datasets than the other two algorithms.

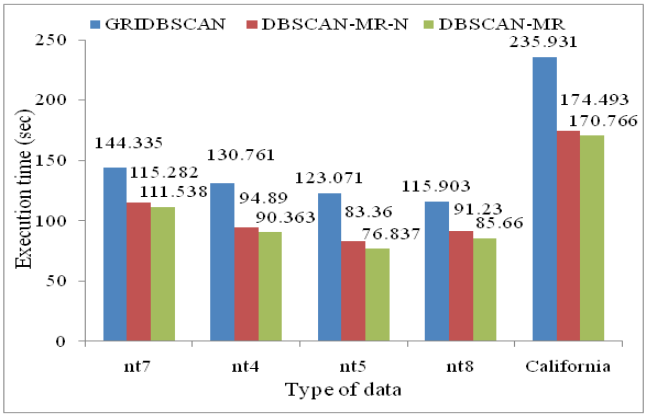


Figure 9. The comparison of execution time for different data sets

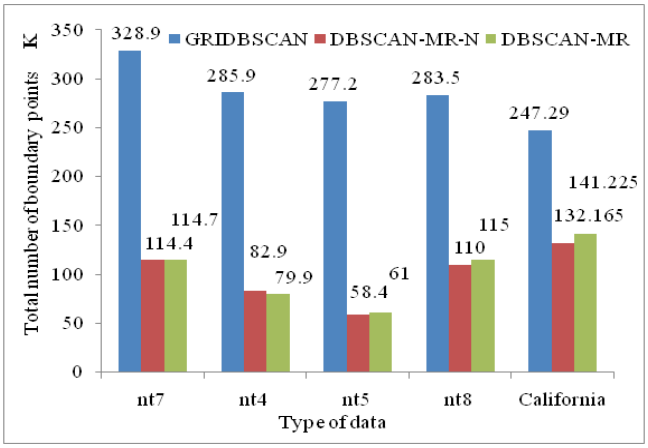


Figure 10. The comparison of total number of boundary points for different data sets (The unit of chart is K).

In the second experiment, different types of datasets, including four synthetic datasets and one real dataset, are processed by these algorithms for comparing the performances. Without loss of generality, the number of nodes is set as 4. As shown in Figure 9, the proposed DBSCAN-MR algorithm is more efficient than other algorithms. It even outperforms GRIDBSCAN algorithm for 21% to 37% on the total execution time. In addition, as shown in Figure 10, the numbers of boundary points of DBSCAN-MR and DBSCAN-MR-N are much less than that of GRIDBSCAN. This illustrates that applying the PRBP process in the partition phase reduces the total number of boundary points for 43% to 77%. Because the

Authorized licensed use limited to: UEL - Universidade Estadual de Londrina. Downloaded on October 21,2026 at 20:04:58 UTC from IEEE Xplore. Restrictions apply.

number of boundary points is significantly reduced, DBSCAN-MR is more efficient than GRIDBSCAN on different types of datasets.

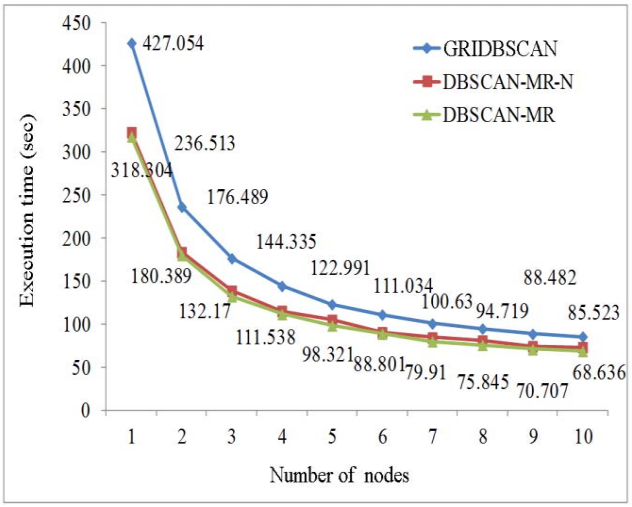


Figure 11. The comparison of execution time for different node count sets.

Finally, we show the advantages of the distributed scheme of our proposed algorithm DBSCAN-MR when it processes the nt7,1000k dataset. As shown in Figure 11, the total execution time drops as the number of nodes is increased from 1 to 7. This shows the merits of the distributed scheme. However, the overheads of disk I/O and message communication retard the reduction rate of the total execution time when the number of nodes is further increased.

In summary, GRIDBSCAN is not efficient because it splits data with lots of redundant boundary points. On the contrary, PRBP algorithm can partition the data set more effectively by taking the data distribution into consideration. With PRBP, the execution time of clustering and merging can be reduced and the load of each node can be balanced. Therefore, DBSCAN-MR-N and DBSAN-MR are much more efficient than GRIDBSCAN.

V. CONCLUSIONS

In this paper, we proposed a new algorithm DBSCAN-MR. It enhanced the performance of DBSCAN by the cloud computing technology. We also designed a data partition algorithm, PRBP, to balance the load of each node and to improve the efficiency of the entire framework. Experimental results verified the high efficiency of DBSCAN-MR over the competitor.

REFERENCES

[1] Fayyad, Usama; Gregory Piatetsky-Shapiro, and Padhraic Smyth (1996). "From Data Mining to Knowledge Discovery in Databases"

[2] Han, P.N., Kamber, M.: Data Mining: Concepts and Techniques, 2ed(2006)

[3] Tan, P.N., Steinbach, M., Kumar, V.: Introduction to Data Mining (2006) .

[4] J. Han, M. Kamber, Data Mining Concepts and Techniques, Morgan Kaufmann Publishers, San Francisco, CA, 2001, pp. 335–391.

[5] Jen-Wei Huang, Su-Chen Lin and Ming-Syan Chen, "DPSP: Distributed Progressive Sequential Pattern Mining on the Cloud," Lecture Notes in Computer Science, 2010, Volume 6119/2010, 27-34

[6] C. Moretti, J. Bulosan, D. Thain, and P. Flynn. All-pairs: An abstraction for data-intensive cloud computing. In IEEE/ACM International Parallel and Distributed Processing Symposium, April 2008.

[7] WHITE, B., YEH, T., LIN, J., AND DAVIS, L. 2010. Web-scale computer vision using mapreduce for multimedia data mining. Proceedings of the Tenth International Workshop on Multimedia Data Mining, 1–10.

[8] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, A density-based algorithm for discovering clusters in large spatial databases with noise, in: Proceedings of Second International Conference on Knowledge Discovery and Data Mining, Portland, OR, 1996, pp. 226–231.

[9] Kryszkiewicz, M., Lasek, P.: TI-DBSCAN: Clustering with DBSCAN by Means of the Triangle Inequality. In: Szczuka, M. (ed.) RSCTC 2010. LNCS, vol. 6086, pp. 60–69. Springer, Heidelberg (2010)

[10] EL-SONBATY, Y., ISMAIL, M. A., AND FAROUK, M. 2004. An efficient density-based clustering algorithm for large databases. In Proceedings of the 16th IEEE International Conference on Tools with Artificial Intelligence (ICTAI'04). IEEE Computer Society, 673–677.

[11] R.T. Ng and J. Han.:" Efficient and Effective Clustering Methods for spatial data mining", Proc. 20th Int. Conf. on Very Large Data Bases, santiago, Chile, 1994, pages 144-155

[12] Ozge Uncu, William A. Gruver, Dilip B. Kotak. GRIDBSCAN: GRId Density-Based Spatial Clustering of Applications with Noise. In IEEE International Conference on Systems, Man and Cybernetics, Taipei, Taiwan, 2006 , pp. 2976-2981

[13] Dean, J., Ghemawat, S.: Mapreduce: Simplified dataprocessing on large clusters. In: Symp. on Operating System Design and Implementation (2004).

[14] Hadoop, http://hadoop.apache.org.

[15] M. Ankerst, M.M. Breunig, H.-P. Kriegel, J. Sander, OPTICS: Ordering points to identify the clustering structure, in: Proceedings of ACM SIGMOD International Conference on Management of Data, Philadelphia, PA, 1999, pp. 49–60

[16] A. Hinneburg, D.A. Keim, An efficient approach to clustering in large multimedia databases with noise, in: Proceedings of 4th International Conference on Knowledge Discovery and Data Mining, New York City, NY, 1998, pp. 58–65.

[17] S. Ma, T.J. Wang, S.W. Tang, D.Q. Yang, J. Gao, A new fast clustering algorithm based on reference and density, in: Proceedings of WAIM, Lectures Notes in Computer Science, 2762, Springer, 2003, pp. 214–225

[18] Cheng, H., Tan, P.-N., Sticklen, J., Punch, W.F.: Recommendation via query cen-tered random walk on k-partite graph. In: Proc. of Intl. Conf. on Data Mining, pp. 457–462 (2007).

[19] Chilson, J., Ng, R., Wagner, A., Zamar, R.: Parallel computation of high dimen-sional robust correlation and covariance matrices. In: Proc. of Intl. Conf. on Knowl-edge Discovery and Data Mining, August 2004, pp. 533–538 (2004).

[20] Kargupta, H., Das, K., Liu, K.: Multi-party, privacy-preserving distributed data mining using a game theoretic framework. In: Proc. of European Conf. on Principles and Practice of Knowledge Discovery in Databases, pp. 523–531 (2007)

[21] V. Gaede and O. G¨unther, "Multidimensional access methods," ACM Comput. Surv., vol. 30, no. 2, pp. 170–231, 1998.

[22] G. Karypis, E. H. Han, and V. Kumar, "CHAMELEON: A hierarchical clustering algorithm using dynamic modeling," Computer, vol. 32, no. 8, pp. 68–75, 1999.

[23] USGS geoname datasets; http://geonames.usgs.gov/.

Authorized licensed use limited to: UEL - Universidade Estadual de Londrina. Downloaded on October 21,2026 at 20:04:58 UTC from IEEE Xplore. Restrictions apply.